

PERSONAL FINANCE ANALYSER

Database project - mysql



FEBRUARY 6, 2025

SHRIHARI JAWALGI
Rooman technologies

PERSONAL FINANCE HEALTH ANALYZER AND OPTIMIZER

Introduction: This project is a MySQL system designed to help individuals analyze and optimize the personal finances. It will include tracking income expenses, savings, investments and debts it will provide actionable insights that will help improving financial wellbeing the system will use

MySQL concepts to generate reports

- Identify Financial trends
- Suggests optimization strategies

PRE- REQUISITES

for this project: in order to there are some pre-requisites

1. MYSQL 8.0 command line client
2. MYSQL 8.0 Workbench CE
3. MYSQL statements
4. MYSQL operations
5. MYSQL clauses
6. MYSQL constraints
7. MYSQL sub-queries
8. MYSQL Joins
9. User permissions (GRANT & REVOKE)
10. Transactions

These are the concepts or technologies which the user needs to-be fundamentally strong

KEY FEATURES OF THIS PROJECT

This project will contain several important features of real-world Finance Management such as

1. Income and expense tracking- it is used to track monthly income sources and categories expenses
2. Savings and investment analysis- Monitoring savings account and investment portfolio along with calculating returns
3. Debt management- it is used to track debts, (credit cards, loans etc.) and calculate interest payments along with suggesting debt re-payment strategies
4. Financial health scoring- It is used to generate a financial health score based on income expenses, investments, saving and debts it is used to assess financial stability
5. Budget Optimization- It is used to suggest budget allocation that are optimal based on previous data
6. User permissions- Used to provide role-based access. It is also used to restrict access on sensitive financial data.

SCHEMA

In order to perform this project, there are various parameters required:

- **Database:** create a new database for this project with the project name- Personal_Finance_Health
- **Tables required:**
 1. Users - this table should contain user's details (User_id, User_name, Role, password)
 2. Income- Used to store income sources of user (Income_id, User_id, source, Amount, Date)
 3. Expense- It is used to collect expense (Expense_id, User_id, Category, Amount, Date)
 4. Savings- Used to store savings details (Saving_id, User_id, Account_type, Amount, Date)
 5. Investments details- (Investment_id, User_id, Type, Amount, Return_rate, Date)
 6. Debts- used to store debts details (Debt_id, User_id, Type, Amount, Interest_rate, Due_date)
 7. FinancialHealth – Used to store financial health scores (Health_id, User_id, Score, Date)With the use of constraint keys, the users table must be linked with all the other table with the use of primary key and foreign key, this is called One-to-Many relationship.
- **Users:**
 - Admin – with admin privileged
 - User – with select only data privileges
- **Implementation:** In order to design the structure for this project and generate analysis based on the data here are the steps of implementation for this project
 - Creatin Database and Creating necessary tables.

INSERTION

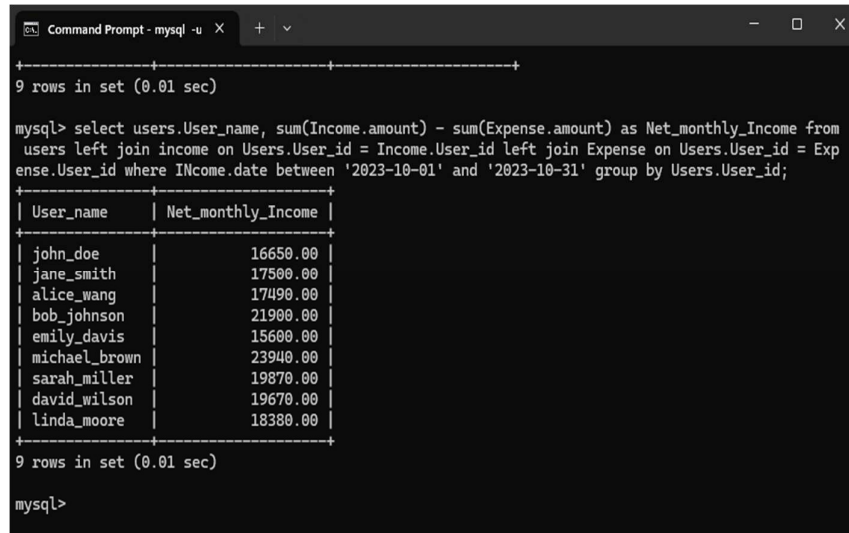
Inserting values more than 100 rows in all 7 tables to perform the analysis. Apart from the auto-increment columns all the columns have to be filled with data

- 1- **Users:** There should be 10 users among those 10- 1 should be named as Admin whereas the rest 9 will be users,
- 2- **Income:** For each user there are different source of income along with profit generated they have been assigned to a particular user id that matches the user ids from the table users
- 3- **Expenses:** This table covers the expenses of all 9 users and it will classify those expenses into various categories
- 4- **Savings:** This table provides the values for each and every saving medium for each user
- 5- **Investment:** This tables used to provide value of each investment done by the user and the return they have received
- 6- **Debts:** this table is used to provide values that are loan information of each user and how much amount they have loaned and at what interest

REQUIREMENTS:

- Calculating **monthly net income** for each user.in order to fetch details for this requirement 3 tables have to be used – user, income and expenses - all the tables have to be joined

MySQL> select users.User_name, sum(Income.amount) - sum(Expense.amount) as Net_monthly_Income from users left join income on Users.User_id = Income.User_id left join Expense on Users.User_id = Expense.User_id where Income.date between '2023-10-01' and '2023-10-31' group by Users.User_id;



```

Command Prompt - mysql -u X
9 rows in set (0.01 sec)

mysql> select users.User_name, sum(Income.amount) - sum(Expense.amount) as Net_monthly_Income from
users left join income on Users.User_id = Income.User_id left join Expense on Users.User_id = Exp
ense.User_id where INcome.date between '2023-10-01' and '2023-10-31' group by Users.User_id;

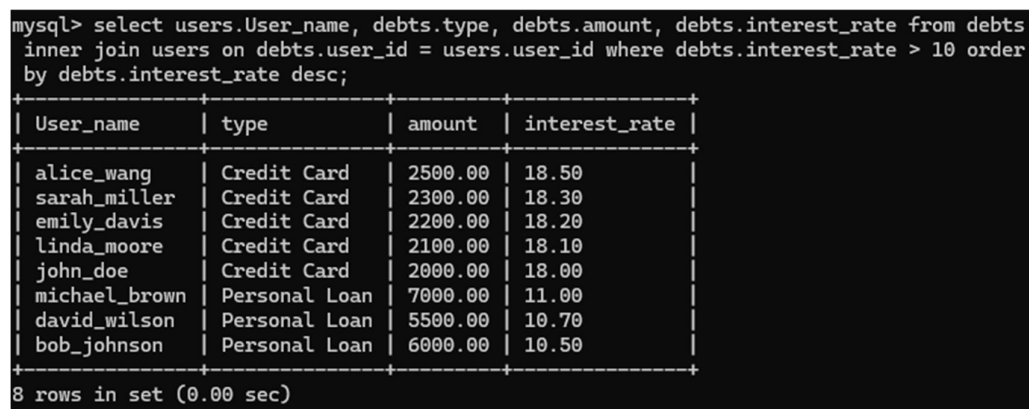
+-----+-----+
| User_name | Net_monthly_Income |
+-----+-----+
| john_doe | 16650.00 |
| jane_smith | 17500.00 |
| alice_wang | 17490.00 |
| bob_johnson | 21900.00 |
| emily_davis | 15600.00 |
| michael_brown | 23940.00 |
| sarah_miller | 19870.00 |
| david_wilson | 19670.00 |
| linda_moore | 18380.00 |
+-----+-----+
9 rows in set (0.01 sec)

mysql>

```

- **Identifying high interest debts:**

MySQL> select users.User_name, debts.type, debts.amount, debts.interest_rate from debts inner join users on debts.user_id = users.user_id where debts.interest_rate > 10 order by debts.interest_rate desc;



```

mysql> select users.User_name, debts.type, debts.amount, debts.interest_rate from debts
inner join users on debts.user_id = users.user_id where debts.interest_rate > 10 order
by debts.interest_rate desc;

+-----+-----+-----+-----+
| User_name | type | amount | interest_rate |
+-----+-----+-----+-----+
| alice_wang | Credit Card | 2500.00 | 18.50 |
| sarah_miller | Credit Card | 2300.00 | 18.30 |
| emily_davis | Credit Card | 2200.00 | 18.20 |
| linda_moore | Credit Card | 2100.00 | 18.10 |
| john_doe | Credit Card | 2000.00 | 18.00 |
| michael_brown | Personal Loan | 7000.00 | 11.00 |
| david_wilson | Personal Loan | 5500.00 | 10.70 |
| bob_johnson | Personal Loan | 6000.00 | 10.50 |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)

```

- **Generate Financial health Score:** Using the data in this requirement the user will calculate financial health score with the help of 4 table – user income, expense and debts
- MYSQL> Select users.user_id, (sum(income.amount) - sum(expense.amount) - sum(debts.amount)) / sum(income.amount) * 100 as Score, Now() from users left join Income on users.user_id = income.user_id left join Expense on users.user_id =

expense.user_id left join debts on users.user_id = debts.user_id group by users.user_id;

```
mysql> Select users.user_id, (sum(income.amount) - sum(expense.amount) -
sum(debts.amount)) / sum(income.amount) * 100 as Score, Now() from user
s left join Income on users.user_id = income.user_id left join Expense o
n users.user_id = expense.user_id left join debts on users.user_id = deb
ts.user_id group by users.user_id
;
```

user_id	Score	Now()
1	-326.000000	2025-02-05 09:57:14
2	-530.000000	2025-02-05 09:57:14
3	-380.250000	2025-02-05 09:57:14
4	-554.736842	2025-02-05 09:57:14
5	-397.142857	2025-02-05 09:57:14
6	-553.674419	2025-02-05 09:57:14
7	-358.037736	2025-02-05 09:57:14
8	-557.308411	2025-02-05 09:57:14
9	-310.693878	2025-02-05 09:57:14
10	NULL	2025-02-05 09:57:14

10 rows in set (0.02 sec)

➤ Budget Optimization:

In this requirement the users will be provided the average spending in based on expenses

MYSQL> SELECT users.user_name, expense.category, avg(expense.amount) as AVG_Spending from expense inner join Users on users.user_id = expense.user_id group by users.user_id Expense.category having AVG_Spending > (Select avg(amount) from expense where category = 'Entertainment');

```
mysql> SELECT users.user_name , expense.category , avg(expense.amount) as AVG_Spending from expense inner
join Users on users.user_id = expense.user_id group by users.user_id , Expense.category having AVG_Spendin
g > (Select avg(amount) from expense where category = 'Entertainment' );
```

user_name	category	AVG_Spending
alice_wang	Rent	1300.000000
alice_wang	Groceries	350.000000
bob_johnson	Rent	1400.000000
bob_johnson	Groceries	320.000000
david_wilson	Rent	1450.000000
david_wilson	Groceries	340.000000
david_wilson	Entertainment	240.000000
emily_davis	Rent	1100.000000
emily_davis	Groceries	280.000000
jane_smith	Rent	1500.000000
jane_smith	Groceries	400.000000
jane_smith	Entertainment	250.000000
john_doe	Rent	1200.000000
john_doe	Groceries	300.000000
linda_moore	Rent	1250.000000
linda_moore	Groceries	310.000000
michael_brown	Rent	1600.000000
michael_brown	Groceries	450.000000
michael_brown	Entertainment	300.000000
sarah_miller	Rent	1350.000000
sarah_miller	Groceries	330.000000
sarah_miller	Entertainment	230.000000

22 rows in set (0.01 sec)

➤ Calculating savings growth rate:

MYSQL> select users.User_name, savings.account_type, (savings.amount) (select sum(amount) from savings where user_id = savings.user_id) * 100 as

SAVINGS_GROWTH_RATE from savings inner join users on savings.user_id = users.user_id;

```
mysql> select users.User_name, savings.account_type, (savings.amount / (select
sum(amount) from savings where user_id = savings.user_id) ) * 100 as SAVINGS_
GROWTH_RATE from savings inner join users on savings.user_id = users.user_id;
```

User_name	account_type	SAVINGS_GROWTH_RATE
alice_wang	Emergency Fund	7.853403
alice_wang	Retirement	3.272251
bob_johnson	Emergency Fund	7.198953
bob_johnson	Retirement	2.879581
david_wilson	Emergency Fund	7.591623
david_wilson	Retirement	3.010471
emily_davis	Emergency Fund	6.282723
emily_davis	Retirement	2.356021
jane_smith	Emergency Fund	9.162304
jane_smith	Retirement	3.926702
john_doe	Emergency Fund	6.544503
john_doe	Retirement	2.617801
linda_moore	Emergency Fund	6.806283
linda_moore	Retirement	2.748691
michael_brown	Emergency Fund	10.471204
michael_brown	Retirement	5.235602
sarah_miller	Emergency Fund	8.507853
sarah_miller	Retirement	3.534031

18 rows in set (0.00 sec)

USER PERMISSIONS:

- Create user 'admin'@'localhost' identified by 'admin123';
- Show grants for 'admin'@'localhost';
- Grant all privileges on personalfinancehealth.* to 'admin'@'localhost' with grant option;
- Select users, host from mysql user;
- create user 'user'@'localhost' identified by 'user123';
- grant select on personalfinancehealth.* to 'user'@'localhost';

TRANSACTIONS

Over the course of the period in the data there are several updates when it comes to income, savings, investments, debts and expenses. In order to update the values, it is safer to update the values inside a transaction. There are 4 transactions to be performed on this project

- A. Record income and update savings.
- B. Debt pay off and update savings.
- C. Transport funds between savings account.
- D. Record investments and update savings.

Demonstrations:

- **RECORD INCOME AND UPDATE SAVINGS:**
Mysql> start transaction;
Mysql> select * from savings;

```
mysql> select * from savings;
```

Saving_id	User_id	Account_type	Amount	Date
1	1	Emergency Fund	5000.00	2023-10-01
2	1	Retirement	2000.00	2023-10-01
3	2	Emergency Fund	7000.00	2023-10-01
4	2	Retirement	3000.00	2023-10-01
5	3	Emergency Fund	6000.00	2023-10-01
6	3	Retirement	2500.00	2023-10-01
7	4	Emergency Fund	5500.00	2023-10-01
8	4	Retirement	2200.00	2023-10-01
9	5	Emergency Fund	4800.00	2023-10-01
10	5	Retirement	1800.00	2023-10-01
11	6	Emergency Fund	8000.00	2023-10-01
12	6	Retirement	4000.00	2023-10-01
13	7	Emergency Fund	6500.00	2023-10-01
14	7	Retirement	2700.00	2023-10-01
15	8	Emergency Fund	5800.00	2023-10-01
16	8	Retirement	2300.00	2023-10-01
17	9	Emergency Fund	5200.00	2023-10-01
18	9	Retirement	2100.00	2023-10-01

mysql> select * from income;

mysql> SAVEPOINT s1;

mysql> insert into income (User_id, source, amount, date) values (1, 'Side-Hustle', 9000, '2023-10-09');

mysql> SAVEPOINT s2;

mysql> update savings set amount=amount + 9000.00 where User_id =1 and Account_type = 'Emergency Funds';

mysql> COMMIT;

```
mysql> select * from income;
```

Income_id	User_id	Source	Amount	Date
1	1	Salary	3000.00	2023-10-01
2	1	Freelance	500.00	2023-10-15
3	1	Bonus	1000.00	2023-10-30
4	2	Salary	4000.00	2023-10-01
5	2	Dividends	200.00	2023-10-10
6	2	Bonus	800.00	2023-10-25
7	3	Salary	3500.00	2023-10-01
8	3	Freelance	600.00	2023-10-20
9	3	Bonus	700.00	2023-10-31
10	4	Salary	4500.00	2023-10-01
11	4	Dividends	300.00	2023-10-15
12	4	Bonus	900.00	2023-10-30
13	5	Salary	3200.00	2023-10-01
14	5	Freelance	400.00	2023-10-10
15	5	Bonus	600.00	2023-10-25
16	6	Salary	5000.00	2023-10-01
17	6	Dividends	250.00	2023-10-15
18	6	Bonus	1200.00	2023-10-31
19	7	Salary	3800.00	2023-10-01
20	7	Freelance	700.00	2023-10-20
21	7	Bonus	800.00	2023-10-30
22	8	Salary	4200.00	2023-10-01
23	8	Dividends	150.00	2023-10-10
24	8	Bonus	1000.00	2023-10-25
25	9	Salary	3600.00	2023-10-01

➤ DEBT PAY OFF AND UPDATE SAVINGS.

mysql> start transaction;

mysql> select * from savings;

```
mysql> select * from savings;
```

Saving_id	User_id	Account_type	Amount	Date
1	1	Emergency Fund	6000.00	2023-10-01
2	1	Retirement	7000.00	2023-10-01
3	2	Emergency Fund	7000.00	2023-10-01
4	2	Retirement	3000.00	2023-10-01
5	3	Emergency Fund	6000.00	2023-10-01
6	3	Retirement	2500.00	2023-10-01
7	4	Emergency Fund	5500.00	2023-10-01
8	4	Retirement	2200.00	2023-10-01
9	5	Emergency Fund	4800.00	2023-10-01
10	5	Retirement	1800.00	2023-10-01
11	6	Emergency Fund	8000.00	2023-10-01
12	6	Retirement	4000.00	2023-10-01
13	7	Emergency Fund	6500.00	2023-10-01
14	7	Retirement	2700.00	2023-10-01
15	8	Emergency Fund	5800.00	2023-10-01
16	8	Retirement	2300.00	2023-10-01
17	9	Emergency Fund	5200.00	2023-10-01
18	9	Retirement	2100.00	2023-10-01

```
18 rows in set (0.00 sec)
```

mysql> select * from debts;

mysql> SAVEPOINT s3;

mysql> update savings set amount = amount - 1000.00 where user_id = 1 and Account_type = 'emergency fund';

mysql> SAVEPOINT s4;

mysql> update debts set amount = amount - 1000.00 where user_id = 1;

mysql> COMMIT;

```
mysql> select * from debts;
```

Debt_id	User_id	Type	Amount	Interest_rate	Due_Date
1	1	Credit Card	1000.00	18.00	2024-01-01
2	1	Student Loan	9000.00	5.00	2025-01-01
3	2	Car Loan	15000.00	6.00	2024-06-01
4	2	Personal Loan	5000.00	10.00	2023-12-01
5	3	Credit Card	2500.00	18.50	2024-02-01
6	3	Student Loan	12000.00	5.50	2025-02-01
7	4	Car Loan	18000.00	6.50	2024-07-01
8	4	Personal Loan	6000.00	10.50	2023-12-15
9	5	Credit Card	2200.00	18.20	2024-01-15
10	5	Student Loan	11000.00	5.20	2025-01-15
11	6	Car Loan	20000.00	7.00	2024-08-01
12	6	Personal Loan	7000.00	11.00	2023-12-20
13	7	Credit Card	2300.00	18.30	2024-02-15
14	7	Student Loan	13000.00	5.30	2025-02-15
15	8	Car Loan	17000.00	6.70	2024-07-15
16	8	Personal Loan	5500.00	10.70	2023-12-10
17	9	Credit Card	2100.00	18.10	2024-01-10
18	9	Student Loan	10500.00	5.10	2025-01-10

```
18 rows in set (0.00 sec)
```

➤ TRANSPORT FUNDS BETWEEN SAVINGS ACCOUNT

mysql> start transaction;

mysql> select * from savings;

mysql> SAVEPOINT s5;

mysql> update savings set amount = amount - 5000.00 where user_id = 1 and Account_type = 'emergency fund';

mysql> SAVEPOINT s6;


```

Mysql> update savings set amount = amount + 5000.00 where user_id = 1 and
Account_type = 'Retirement';
Mysql> COMMIT;

```

```
mysql> select * from savings;
```

Saving_id	User_id	Account_type	Amount	Date
1	1	Emergency Fund	6000.00	2023-10-01
2	1	Retirement	7000.00	2023-10-01
3	2	Emergency Fund	7000.00	2023-10-01
4	2	Retirement	3000.00	2023-10-01
5	3	Emergency Fund	6000.00	2023-10-01
6	3	Retirement	2500.00	2023-10-01
7	4	Emergency Fund	5500.00	2023-10-01
8	4	Retirement	2200.00	2023-10-01
9	5	Emergency Fund	4800.00	2023-10-01
10	5	Retirement	1800.00	2023-10-01
11	6	Emergency Fund	8000.00	2023-10-01
12	6	Retirement	4000.00	2023-10-01
13	7	Emergency Fund	6500.00	2023-10-01
14	7	Retirement	2700.00	2023-10-01
15	8	Emergency Fund	5800.00	2023-10-01
16	8	Retirement	2300.00	2023-10-01
17	9	Emergency Fund	5200.00	2023-10-01
18	9	Retirement	2100.00	2023-10-01

18 rows in set (0.00 sec)

➤ RECORD INVESTMENTS AND UPDATE SAVINGS.

```

Mysql> start transaction;
Mysql> select * from savings;
Mysql> select * from investment;
Mysql> SAVEPOINT s7;
Mysql> update savings set amount = amount - 2000.00 where user_id = 1 and
Account_type = 'Emergency fund';
Mysql> SAVEPOINT s8;
Mysql> insert into investments (user_id, type, amount, Return_rate, date) values
(1,'Mutual Funds',2000.00, 600, '2023-10-13');
Mysql> COMMIT;

```

```
mysql> select * from investments;
```

ERROR 1146 (42S02): Table 'personalfinancehealth.investments' doesn't exist

```
mysql> select * from investment;
```

Investment_id	User_id	Type	Amount	Return_rate	Date
1	1	Stocks	10000.00	8.50	2023-10-01
2	1	Bonds	5000.00	3.00	2023-10-01
3	2	Mutual Funds	15000.00	6.00	2023-10-01
4	2	Real Estate	20000.00	5.00	2023-10-01
5	3	Stocks	12000.00	8.00	2023-10-01
6	3	Bonds	6000.00	3.50	2023-10-01
7	4	Mutual Funds	18000.00	6.50	2023-10-01
8	4	Real Estate	25000.00	5.50	2023-10-01
9	5	Stocks	11000.00	8.20	2023-10-01
10	5	Bonds	5500.00	3.20	2023-10-01
11	6	Mutual Funds	20000.00	7.00	2023-10-01
12	6	Real Estate	30000.00	6.00	2023-10-01
13	7	Stocks	13000.00	8.30	2023-10-01
14	7	Bonds	7000.00	3.30	2023-10-01
15	8	Mutual Funds	17000.00	6.70	2023-10-01
16	8	Real Estate	22000.00	5.70	2023-10-01
17	9	Stocks	10500.00	8.10	2023-10-01
18	9	Bonds	5200.00	3.10	2023-10-01
19	1	Mutual Funds	2000.00	600.00	2023-10-13
20	1	Mutual Funds	2000.00	600.00	2023-10-13

20 rows in set (0.00 sec)

Conclusion

1. Using this project the user can maintain a good personal finance score hereby helping them while making future financial decisions and maintain a good track of all the future assets and liabilities. With the use of table such as income, expenses, debts, saving, investments the user generated the financial health table which provided them the score of their personal finance.
2. Many analytics were produced apart from that many other analytics can be performed based on the data such as finding the debt-to-income ratio (debt burden analysis, investment performance analysis, monthly expense trends and so for). The code has to be stored in .sql file. Prepare a report of the findings along with the sql file and it will be hosted in GITHUB.

Enhancements

- The project can be enhanced with some more features which are in nature such as integrating with API to fetch real life data
- Data visualization to create interactive dashboard and reports
- Machine learning – implementing machine learning models to predict financial health using certain algo and provide some recommendations.

Impact of the project

This project demonstrate proficiency in mysql and also the ability to solve real world problems using database systems. It is unique and practical addition for a portfolio and will help the user advance in the field of database systems